MODULE 18 · DOMAIN 3 - DEPLOYMENT, PROVISIONING, AND AUTOMATION

CloudFormation: Template Anatomy

Sections, functions, and the five resource attributes that decide what happens when things go wrong.

DOMAIN 3 of 5 EXAM WEIGHT 22% MODULE 18 of 35

admin@sthithapragna.com

The template says what you want. The **resource attributes** say what happens when creation fails, when an update replaces something, and when the stack is deleted.

The template sections

SECTION	WHAT IT DOES
AWSTemplateFormatVersion	Optional, and effectively always 2010-09-09
Description, Metadata	Documentation, and console grouping of parameters
Parameters	Inputs supplied at deployment time
Rules	Validate parameter combinations before anything is created
Mappings	Static lookup tables, most often keyed by Region
Conditions	Named booleans that switch resources and properties on and off
Transform	Macros: SAM, AWS::Include, AWS::LanguageExtensions
Resources	The only required section
Outputs	Values to return, and optionally export for other stacks

ON THE EXAM

Resources is the only mandatory section. A template with one resource and nothing else is completely valid.

Parameters

- Types include **String, Number, CommaDelimitedList**, and **AWS-specific types** such as AWS::EC2::Subnet::Id.
- AWS-specific types give a **validated dropdown in the console** and fail fast on a bad value, rather than failing halfway through a deployment.
- **SSM parameter types** read a value from Parameter Store at deployment time, which is how you avoid hard-coded AMI IDs.
- **NoEcho** masks a value in the console, CLI and API. It does **not** encrypt it, and anyone who can read the template still sees a default.

> ON THE EXAM
>
> Constraints belong in the template. AllowedValues, AllowedPattern, MinLength and MaxLength turn a runtime failure into an immediate rejection.

Pseudo parameters

- **AWS::Region**, **AWS::AccountId**, **AWS::StackName**, **AWS::StackId**. Supplied by CloudFormation, never declared.
- **AWS::Partition** and **AWS::URLSuffix** make a template work in GovCloud and China as well as commercial Regions.
- **AWS::NoValue** removes a property entirely, which is how a condition makes a property optional rather than empty.
- **AWS::NotificationARNs** gives the SNS topics associated with the stack.

> ON THE EXAM
>
> Fn::If with AWS::NoValue is the idiom for "include this property only sometimes". Setting it to an empty string is not the same thing.

Intrinsic functions worth knowing

FUNCTION	WHAT IT RETURNS
`Ref`	A parameter's value, or a resource's default identifier
`Fn::GetAtt`	A named attribute of a resource, such as an ARN or a DNS name
`Fn::Sub`	A string with ${variables} substituted. The readable choice
`Fn::Join`	Values joined by a delimiter. Older and harder to read than Sub
`Fn::FindInMap`	A value from the Mappings section
`Fn::ImportValue`	A value exported by another stack
`Fn::GetAZs, Fn::Select, Fn::Split`	Availability Zones, and list manipulation
`Fn::If, Fn::Equals, Fn::And, Fn::Or, Fn::Not`	Condition logic

ON THE EXAM

Fn::Sub replaced Fn::Join for almost all practical purposes. Both appear in exam templates, so read both.

Ref or GetAtt?

REF RETURNS

The default identifier

- For a parameter: its value
- For an EC2 instance: the instance ID
- For an S3 bucket: the bucket name
- For a security group: the group ID

GETATT RETURNS

A specific named attribute

- Bucket ARN, DomainName, WebsiteURL
- Instance PrivateIp, PublicDnsName
- Load balancer DNSName, CanonicalHostedZoneID
- Whatever the resource documentation lists

THE TIEBREAKER

Ref returns one thing per resource type and you have to know which. GetAtt returns whatever you name, if the resource publishes it.

Conditions and Mappings

- A **Condition** is a named boolean, evaluated once at deployment. Attach it to a resource with the **Condition** attribute.
- The usual pattern: one template for all environments, with production-only resources behind a condition.
- **Mappings** are static lookups, most often **Region to AMI ID** or environment to instance size.
- Mappings cannot use parameters or functions as keys unless you add the **AWS::LanguageExtensions** transform.

> ON THE EXAM
>
> An SSM parameter type is usually better than a Region-to-AMI mapping. The mapping goes stale; Parameter Store does not.

Dynamic references

- **{{resolve:ssm:name:version}}** pulls a plaintext value from Parameter Store at deployment time.
- **{{resolve:secretsmanager:...}}** pulls a secret, so a password never appears in the template or in a parameter.
- This is the correct answer to "how do we avoid putting the database password in the template?"
- The value is resolved at **deployment**, so rotating the secret afterwards does not update the deployed resource by itself.

> ON THE EXAM
>
> A parameter with NoEcho hides the value in the console. A dynamic reference means the value was never in the template at all.

Outputs and exports

- **Outputs** return values from a stack. Adding **Export** makes one importable by other stacks in the same account and Region.
- **Export names must be unique per account per Region**, and they do not cross Region or account boundaries.
- **You cannot delete a stack whose exports are being imported**, and you cannot change the exported value while it is in use.
- That rigidity is why **nested stacks** are often preferred for tightly coupled resources. Module 19 compares them.

> ON THE EXAM
>
> Exports create a hard dependency that is invisible from the importing stack's template. That is the cost of cross-stack references.

The five resource attributes

ATTRIBUTE	WHAT IT CONTROLS
`DependsOn`	Explicit ordering, when CloudFormation cannot infer the dependency
`DeletionPolicy`	What happens to the resource when it is removed from the stack
`UpdateReplacePolicy`	What happens to the OLD resource when an update replaces it
`CreationPolicy`	Wait for signals before declaring the resource created
`UpdatePolicy`	How to update an Auto Scaling group, Lambda alias or cache cluster

ON THE EXAM

This table is the module. Almost every non-trivial CloudFormation question turns on one of these five, and most turn on the middle two.

DeletionPolicy

- **Delete** is the default for most resources. **Retain** keeps the resource and removes it from the stack's management.
- **Snapshot** takes a final snapshot first. Supported by resources that have snapshots: RDS, EBS volumes, ElastiCache, Redshift.
- **RetainExceptOnCreate** retains on delete, but cleans up if the **initial creation** fails. Avoids orphans from a failed first deploy.
- RDS DB instances and clusters that do not specify a cluster identifier **default to Snapshot**, not Delete.

> ON THE EXAM
>
> Retain on a database is the difference between deleting a stack and deleting a company's data. Set it deliberately, not by accident.

DeletionPolicy or UpdateReplacePolicy?

DELETIONPOLICY FIRES WHEN

The resource leaves the stack

- The stack is deleted
- The resource is removed from the template
- Values: Delete, Retain, Snapshot, RetainExceptOnCreate

UPDATEREPLACEPOLICY FIRES WHEN

An update replaces the resource

- A property change requires replacement
- The OLD resource is what this governs
- Same values, and it defaults to Delete

THE TIEBREAKER

Set both on anything holding data. A DeletionPolicy of Retain does nothing if an update quietly replaces the database and deletes the old one.

DependsOn, and why implicit ordering usually suffices

- CloudFormation **infers** ordering from Ref and GetAtt. If A references B, B is created first, automatically.
- **DependsOn** is for dependencies with no reference between them, such as a route needing the internet gateway attached.
- Over-using DependsOn **serialises the deployment**, turning parallel creation into a slow chain.
- The classic legitimate case: **AWS::EC2::Route depending on AWS::EC2::VPCGatewayAttachment**.

> ON THE EXAM
>
> If a template is slow to deploy, look for DependsOn chains. CloudFormation parallelises aggressively unless you tell it not to.

CreationPolicy and cfn-signal

- **CreationPolicy** makes CloudFormation wait for a number of success signals within a timeout before marking a resource created.
- The instance sends them with **cfn-signal**, usually at the end of user data, once the application is genuinely ready.
- Without it, an EC2 instance is **CREATE_COMPLETE the moment it launches**, long before the application works.
- Applies to EC2 instances, Auto Scaling groups, WaitConditions and AppStream fleets.

> ON THE EXAM
>
> "The stack reports success but the application is not running" is a missing CreationPolicy. Launched is not the same as ready.

UpdatePolicy

- For Auto Scaling groups: **AutoScalingRollingUpdate**, **AutoScalingReplacingUpdate**, **AutoScalingScheduledAction**.
- **AutoScalingInstanceRefresh**, added July 2026, is the modern option. Module twelve covered it: checkpoints, launch-before-terminate, alarm rollback.
- It is **mutually exclusive with AutoScalingRollingUpdate**, which suspends scaling policies and ignores your maintenance policy.
- Also covers **Lambda alias** deployments through CodeDeploy, and ElastiCache online resharding.

> ON THE EXAM
>
> The old rolling update terminates before launching, so capacity dips. The new instance refresh policy can launch first. That is the deciding difference.

What an update actually does

- **Update with no interruption**: the resource keeps running and keeps its physical ID. Changing a Lambda function's memory, for example.
- **Update with some interruption**: the resource is briefly unavailable but keeps its ID. Changing an EC2 instance type.
- **Replacement**: a **new resource is created and the old one deleted**. A new physical ID, and anything depending on it updates too.
- Which one applies is documented **per property** in the resource reference, and a change set tells you before you commit.

> ON THE EXAM
>
> Replacement is where data disappears. Changing an RDS instance's DBInstanceIdentifier replaces the database, and without an UpdateReplacePolicy the old one is deleted.

Stack policies

- A **stack policy** is JSON attached to a stack that denies update actions on chosen resources.
- The usual pattern: **deny Update:Replace and Update:Delete on the database**, allow everything else.
- Once applied, a stack policy **cannot be removed**, only replaced with another one.
- To update a protected resource deliberately, supply a **temporary overriding policy** with the update request.

> ON THE EXAM
>
> A stack policy protects against updates. A DeletionPolicy protects against deletion. They cover different accidents and are not substitutes.

Transform, and language extensions

- **AWS::Serverless** is SAM, a shorthand for Lambda, API Gateway and DynamoDB that expands into full CloudFormation.
- **AWS::Include** inserts a template snippet from S3, for reusable boilerplate.
- **AWS::LanguageExtensions** adds **Fn::ForEach**, **Fn::Length**, **Fn::ToJsonString**, and a default value for Fn::FindInMap.
- Language extensions work only in **Resources, Conditions and Outputs**, and must be listed **before AWS::Serverless**.

> ON THE EXAM
>
> Fn::ForEach removes the copy-paste template. Twenty near-identical resources become one definition and a list.

CloudFormation Hooks

- A **Hook** inspects resource configuration **before** provisioning and can **warn** or **fail** the operation.
- Four kinds: **proactive controls** from the Control Tower Control Catalog, **Guard Hooks**, **Lambda Hooks**, and custom Hooks.
- Managed proactive controls arrived in **September 2025**, so common checks need no custom code at all.
- This is a **preventive** control. AWS Config is detective, catching the same problem after it exists.

> ON THE EXAM
>
> Warn mode first, then fail. It lets you measure how often a control would have blocked a deployment before it actually does.

A CloudFormation stack manages an application and its RDS database. The database resource has DeletionPolicy set to Retain. A developer changes a property that requires replacement of the DB instance and applies the update.

After the update, the original database no longer exists. The team expected Retain to protect it.

What happened, and what should have been set?

A DeletionPolicy only applies to stack deletion and resource removal. UpdateReplacePolicy: Retain governs replacement

B A stack policy was required to prevent the update

C DeletionPolicy: Retain is not supported on RDS instances

D The update should have been made through a change set

◆ PAUSE AND DECIDE

What happened, and what should have been set?

A UpdateReplacePolicy governs replacement

The two attributes cover different events. Replacement deletes the old resource under UpdateReplacePolicy, which defaults to Delete.

B A stack policy

Genuinely would have blocked the update, and it is a different control. The question asks what governs the old resource's fate, not how to block the change.

C Retain unsupported on RDS

It is fully supported. RDS instances without a cluster identifier even default to Snapshot rather than Delete.

D Use a change set

A change set would have warned that replacement was coming, which is valuable. It does not change what happens to the old resource once you proceed.

THE SIGNAL

Set both attributes on anything holding data. This is the most expensive gap in the module and it looks like a single setting.

A CloudFormation stack creates an Auto Scaling group of application servers. The stack reports CREATE_COMPLETE within four minutes, but the application takes around eight minutes to install and start on each instance.

A pipeline runs smoke tests immediately after the stack completes, and they fail intermittently.

What should be added to the template?

A A DependsOn attribute linking the Auto Scaling group to the load balancer

B A CreationPolicy with a ResourceSignal, and cfn-signal at the end of user data

C An UpdatePolicy with AutoScalingRollingUpdate

D A WaitCondition with a fixed 10-minute timeout and no signal

◆ PAUSE AND DECIDE

What should be added to the template?

A DependsOn the load balancer

Changes ordering between resources. The problem is not order; it is that an instance counts as created before it is ready.

B CreationPolicy plus cfn-signal

The stack waits for each instance to signal success from its own user data, so CREATE_COMPLETE means the application is actually running.

C UpdatePolicy rolling update

Governs updates, not creation. The scenario describes the initial deployment reporting success too early.

D A fixed 10-minute wait

The tempting shortcut. It waits whether or not anything worked, so it turns an intermittent failure into a slower intermittent failure.

THE SIGNAL

Launched is not ready. CreationPolicy is the only mechanism that makes CloudFormation's definition of complete match the application's.

A template must not contain the database master password. The password is already stored in AWS Secrets Manager. The team currently passes it as a template parameter with NoEcho set to true.

A security review objects that the password still appears in the pipeline's parameter file and in CloudTrail-visible API calls.

What should the team change?

A Keep the parameter and encrypt the parameter file at rest

B Use a dynamic reference: {{resolve:secretsmanager:...}} directly in the resource property

C Change the parameter type to AWS::SSM::Parameter::Value<String>

D Move the password into a Mappings section

◆ PAUSE AND DECIDE

What should the team change?

A. Encrypt the parameter file

The value still travels through the pipeline and the API call. Encrypting where it rests does not remove it from where it moves.

B. Dynamic reference to Secrets Manager

CloudFormation resolves the secret itself at deployment. The password is never in the template, the parameter file or the API call.

C. SSM parameter type

Better than a plaintext parameter, and Parameter Store is not where the secret lives, and this type reads plaintext values.

D. Mappings

Mappings are plaintext in the template, which is strictly worse than a NoEcho parameter.

THE SIGNAL

NoEcho hides a value that is present. A dynamic reference means the value was never present. The review objected to presence, not visibility.

> A template creates 24 nearly identical SQS queues, one per business region, differing only in name. Adding a property to all of them means editing 24 blocks by hand, and a recent change was applied to 23 of them.
>
> The team wants one definition driven by a list.

What should the team use?

A. Fn::ForEach with the AWS::LanguageExtensions transform

B. A nested stack containing one queue, instantiated 24 times

C. A Mappings section keyed by region name

D. A custom resource backed by a Lambda function that creates the queues

◆ PAUSE AND DECIDE

What should the team use?

A Fn::ForEach

Exactly what it was built for: one definition, a list, and CloudFormation generates the resources. A property change is edited once.

B 24 nested stacks

Solves the duplication and replaces it with 24 stacks to deploy, monitor and update. Far heavier than the problem requires.

C Mappings

Mappings supply values to resources. They do not create resources, so all 24 blocks would still exist.

D Lambda custom resource

Moves the resources outside CloudFormation's model, so drift detection and deletion no longer see them properly.

THE SIGNAL

The clue is 23 of 24. Copy-paste templates fail by omission, and that failure mode is what a loop removes.

> A platform team covers the whole organisation. No CloudFormation deployment may create an unencrypted S3 bucket, or an unencrypted EBS volume. They want the deployment to fail, not to be remediated afterwards.
>
> They also want to trial the control without blocking teams first.

What should the team implement?

A AWS Config rules with automatic SSM Automation remediation

B CloudFormation Hooks using managed proactive controls, in warn mode first and then fail mode

C A stack policy applied to every stack

D An SCP denying s3:CreateBucket without encryption

◆ PAUSE AND DECIDE

What should the team implement?

A. Config rules with remediation

Detective, then corrective. The resource is created non-compliant and fixed afterwards, which the requirement explicitly rejects.

B. Hooks with managed controls

Preventive: the deployment fails before provisioning. Managed controls need no custom code, and warn mode allows a safe trial.

C. Stack policies everywhere

Stack policies deny updates to existing resources in one stack. They cannot evaluate whether a new resource is compliant.

D. SCP on s3:CreateBucket

Genuinely preventive and covers only the API-level condition, with no equivalent for EBS here, and no warn mode to trial with.

THE SIGNAL

Two clauses: fail rather than remediate, and trial before enforcing. Warn mode is the second one, and only Hooks offer it.

The five things to carry forward

- **DeletionPolicy and UpdateReplacePolicy cover different events.** Set both on anything holding data.
- **CreationPolicy plus cfn-signal** makes CREATE_COMPLETE mean the application is actually running.
- **Ref gives one default value per resource type; GetAtt gives whatever you name.**
- **Dynamic references keep secrets out of the template entirely.** NoEcho only hides a value that is present.
- **Hooks are preventive; Config is detective.** Fn::ForEach removes the copy-paste template.

> ON THE EXAM
>
> Next: CloudFormation operations - change sets, drift, nested stacks, StackSets and failed rollbacks.